CJAVA

siempre para apoyarte

Visión

Poder aportar al desarrollo del País usando tecnología Java.

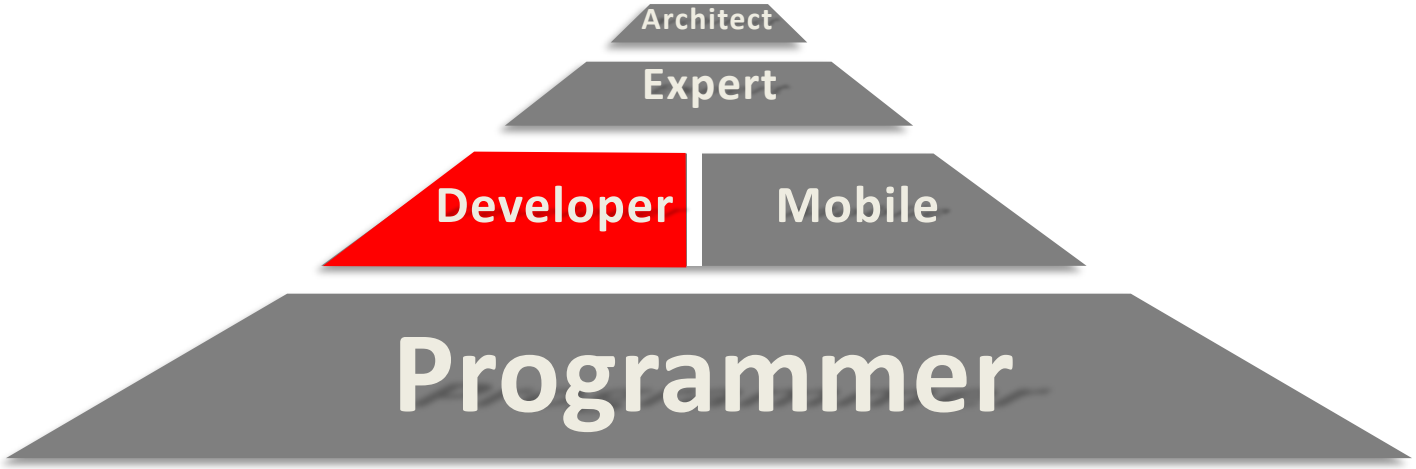
Edwin Maravi
emaravi@cjavaperu.com

CJAVA

Misión

Nuestro equipo trabaja para integrar la tecnología Java en la sociedad como solución a todas sus necesidades.

La Carrera



Temas:

- JPA. Configuración y consultas.

Persistencia con JPA

Edwin Maravi

emaravi@cjavaperu.com

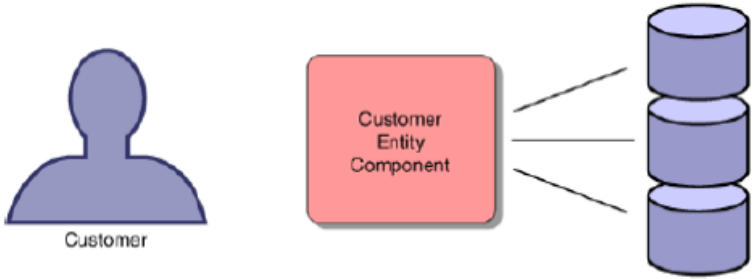
CJAVA
siempre para apoyarte

La API de Persistencia de Java.

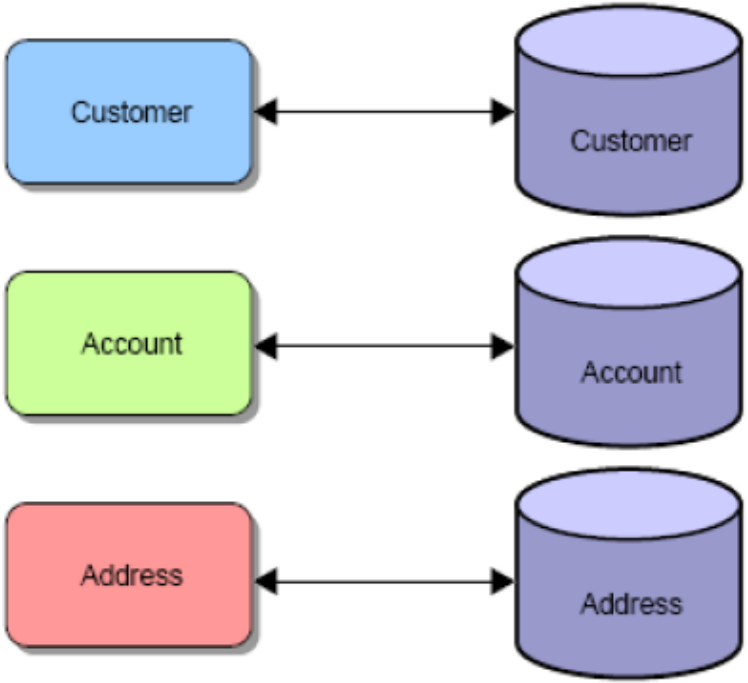
- Reemplaza los EJBs llamados Entity Beans con clases Entidad que no son EJBs.
- Es una API standard para especificar información de mapeo de Objetos a Tablas Relacionales.
- Se puede usar con o sin un Application Server de Java EE.
- Hay dos tipos de persistencia:
  - Container-Managed Persistence
  - Application-Managed Persistence

Software de Mapeo Objeto-Relacional (ORM).

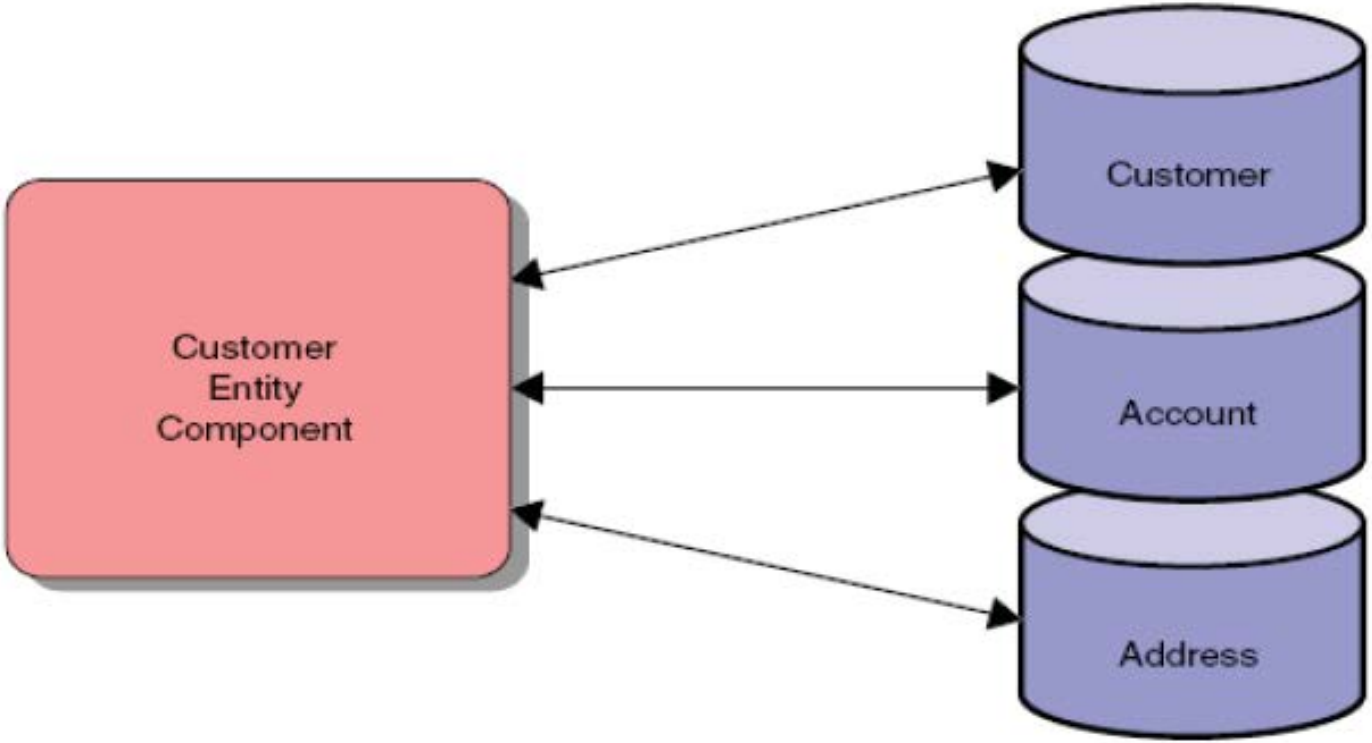
- Proporciona la conversión automática de tablas relacionales a objetos en memoria.
- Algunos ejemplos son Oracle Toplink, GlassFish, Toplink Essentials, BEA Systems Kodo y Hibernate.



Forma Básica de ORM.



Ejemplo de Forma Avanzada de ORM.



CJAVA
siempre para apoyarte

Java Persistence API y ORM.

- La API de Persistencia de Java especifica las características de tecnologías existentes de ORM en la tecnología Java EE.
- Es solamente una especificación, no un software ORM utilizable.
- Se requiere una implementación particular.
- Sun Application Server (GlassFish) utiliza Toplink Essentials (Implementación de Referencia) como default.

CJAVA
siempre para apoyarte

Declaración de las Clases Entidad.

- Se codifican como clases standard de Java con las siguientes características:
  - Marcadas con la anotación javax.persistence.Entity en la clase o declaradas como "Entities" en un Deployment Descriptor.
  - Deben ser clases públicas.
  - Tener un constructor sin argumentos con acceso public o protected.
  - Que se puedan extender, es decir, que no sean finales.
  - Que implementen la interfaz serializable si van a ser entregadas vía return por EJBs de sesión remotos.
  - No pueden ser clases internas (Inner Classes).

Ejemplo de una Clase Entidad.

```
import java.io.Serializable;
import javax.persistence.*;
@Entity @Table(name = "TABLE1")
public class MyEntity implements Serializable {
        @Id @Column(name = "ID") private int id;
        @Column(name = "MSG") private String message;
        protected MyEntity() { }
        public MyEntity(int id, String message) {
        this.id = id;
        this.message = message;
        }
        public int getId() { return id; }
        public String getMessage() { return message; }
        public void setMessage(String message) {
        this.message = message; }
        }
}
```

Mapeo de Default.

- El nombre de la tabla es por default igual a la de la Clase Entidad.
- Se puede cambiar con la anotación @Table.
  - @Entity
  - @Table(name = "Customer")
  - public class Cliente { L }

Elemento en Memoria	Elemento en Base de Datos
Clase Entidad	Tabla
Atributo en Clase Entidad	Columna de la Tabla
Instancia de la Clase Entidad	Fila de la Tabla

Mapeo de Default (2).

- El nombre de la propiedad o atributo de la Clase Entidad es por default igual al nombre de la columna de la tabla.
- Se puede cambiar con la anotación @Column.

```
@Entity
@Table(name = "Customer")
public class Cliente {
@Column(name="nombre")
private String name;
}
```

Campos Persistentes vs. Propiedades Persistentes.

- El estado de las Clases Entidad se sincroniza con la base de datos.
- El estado de una Clase Entidad se obtiene de sus variables de instancia (campos) o de sus propiedades (variables más métodos getters y setters).
- La obtención del estado se determina por la colocación de las anotaciones.
- No se pueden tener ambos casos.

Obtención de Estado vía Campos Persistentes.

Cuando el acceso es vía Campos Persistentes, el contenedor obtiene el estado leyendo directamente las variables de la clase.

- Las variables no pueden ser públicas.
- No deben ser leídas directamente por el cliente.
- Todas las variables son "persistidas" a menos que se declaren como transient de java o se les ponga la anotación @Transient.
- Por ejemplo:

```
@Id @Column(name = "ID") private int id;
@Column(name = "MSG") private String message;
public int getId() { return id; }
public void setId(int id) { this.id = id; }
public String getMessage() { return message; }
public void setMessage(String message) { this.message = message; }
```

Obtención de Estado vía Propiedades Persistentes.

Cuando el acceso es vía Propidades Persistentes, el contenedor obtiene el estado usando los métodos getter.

- Los métodos deben ser públicos o portected.
- Deben seguir las convenciones de nombres de los Java Beans.
- Las anotaciones deben ir en los métodos getters.
- Por ejemplo:

```
private int id;
private String message;
@Id @Column(name = "ID") public int getId() { return id; }
public void setId(int id) { this.id = id; }
@Column(name = "MSG")
public String getMessage() { return message; }
public void setMessage(String message) { this.message = message; }
```

CJAVA
siempre para apoyarte

Tipos de Datos Persistentes.

- Primitivos de Java.
- Objetos de clases Wrapper de Java.
- java.lang.String
- byte[ ] y Byte[ ]
- char[ ] y Character[ ]
- Cualquier tipo serializable como:
  - java.util.Date
  - java.sql.Date
  - java.sql.TimeStamp

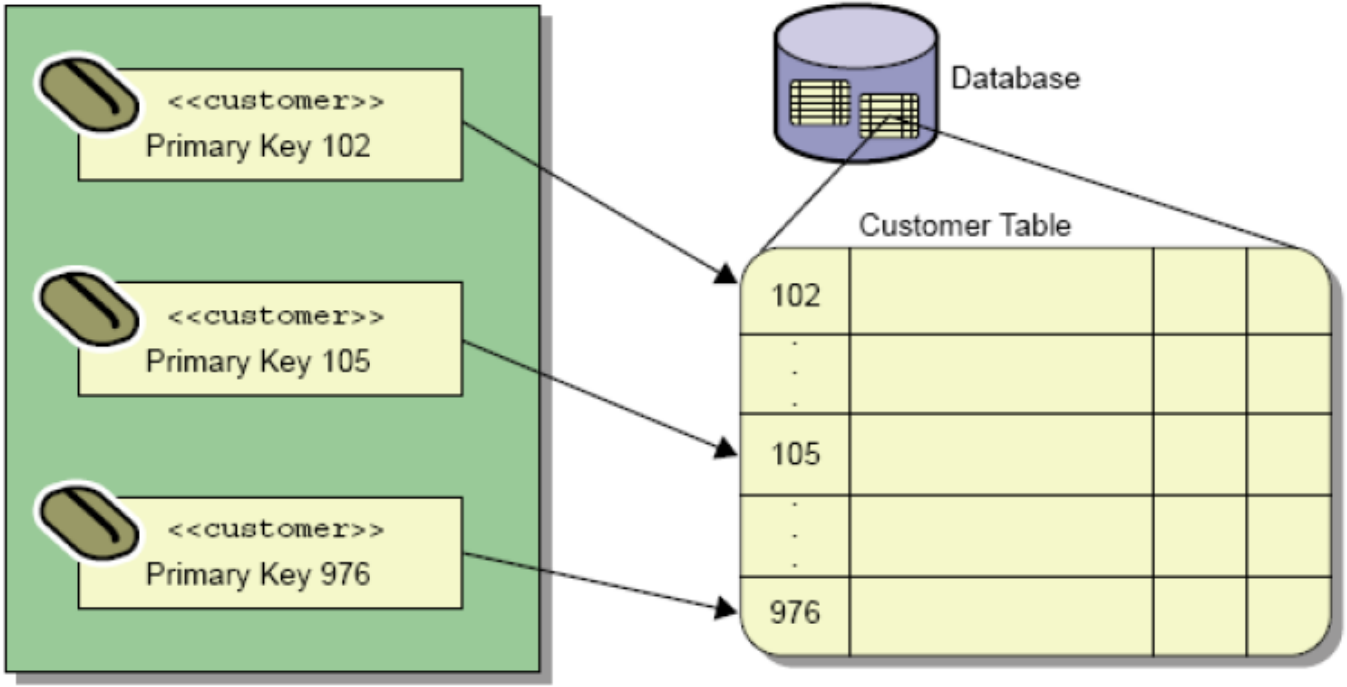
CJAVA
siempre para apoyarte

Concepto de Llave Primaria.

- Cada objeto de Clase Entidad se distingue por tener una llave primaria.
- La llave primaria proporciona identidad a la Clase Entidad.
- Típicamente son Strings o enteros, pero pueden también ser objetos de otras clases que corresponden a varias columnas de la tabla que representan.
- Cada Clase Entidad debe tener una llave primaria.
- Se marcan con la anotación @Id.
- Pueden ser autoincrementables.
- Por ejemplo:

```
@Id @GeneratedValue(strategy = GenerationType.AUTO)
@Column(name = "ID", nullable = false)
private int id;
```

Relación entre Entidades y Filas de Tabla.



CJAVA
siempre para apoyarte

Otros Conceptos de la API de Persistencia.

- Unidad de Persistencia (Persistence Unit).
- Administrador de Entidad (Entity Manager).
- Contexto de Persistencia (Persistence Context).
- Identidad de Persistencia (Persistence Identity).

  Es la llave primaria.

CJAVA
siempre para apoyarte

Persistence Unit.

- Una Unidad de Persistencia en una colección de Clases Entidad representadas por un archivo llamado persistence.xml contenido dentro de un archivo EAR, JAR o WAR de una aplicación Java EE.
- Una Unidad de Persistencia define que Clases Entidad son administradas por un Entity Manager.
- Está limitada a un solo DataSource.

El Archivo persistence.xml.

- Configura que clases constituyen una Unidad de Persistencia.
- Define la base de la Unidad de Persistencia, es decir el archivo al que pertenece.
- Especifica el Data Source utilizado.
- Ejemplo:

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="1.0" xmlns="http://java.sun.com/xml/ns/persistence">
<persistence-unit name="BrokerTool-ejb" transaction-type="JTA">
<jta-data-source>StockMarket</jta-data-source>
<jar-file>BrokerLibrary.jar</jar-file>
<properties/>
</persistence-unit>
</persistence>
```

CJAVA
siempre para apoyarte

El Persistence Context.

- Se interpreta como una copia activa de una Unidad de Persistencia.
- Se pueden tener varios Contextos de Persistencia que usan la misma Unidad de Persistencia activos al mismo tiempo.
- El Contexto de Persistencia:
  - Típicamente dura lo mismo que una transacción.
  - Limita las instancias de Entidades a una sola instancia por Identidad de Persistencia.
  - Tiene una API para su manejo, conocida como el Entity Manager.

CJAVA
siempre para apoyarte

El Entity Manager.

- Proporciona los métodos para controlar eventos en un Contexto de Persistencia y el ciclo de vida de las instancias de Clases Entidad en un Contexto de Persistencia.
- El Entity Manager:
  - Proporciona operaciones como flush(), find() y createQuery() para controlar un Contexo de Persistencia.
  - Reemplaza la funcionalidad de las interfaces home de los EJBs de entidad anteriores.
  - Se obtiene usando anotaciones en clases manejadas por el contenedor:

```
@PersistenceContext private EntityManager em;
```

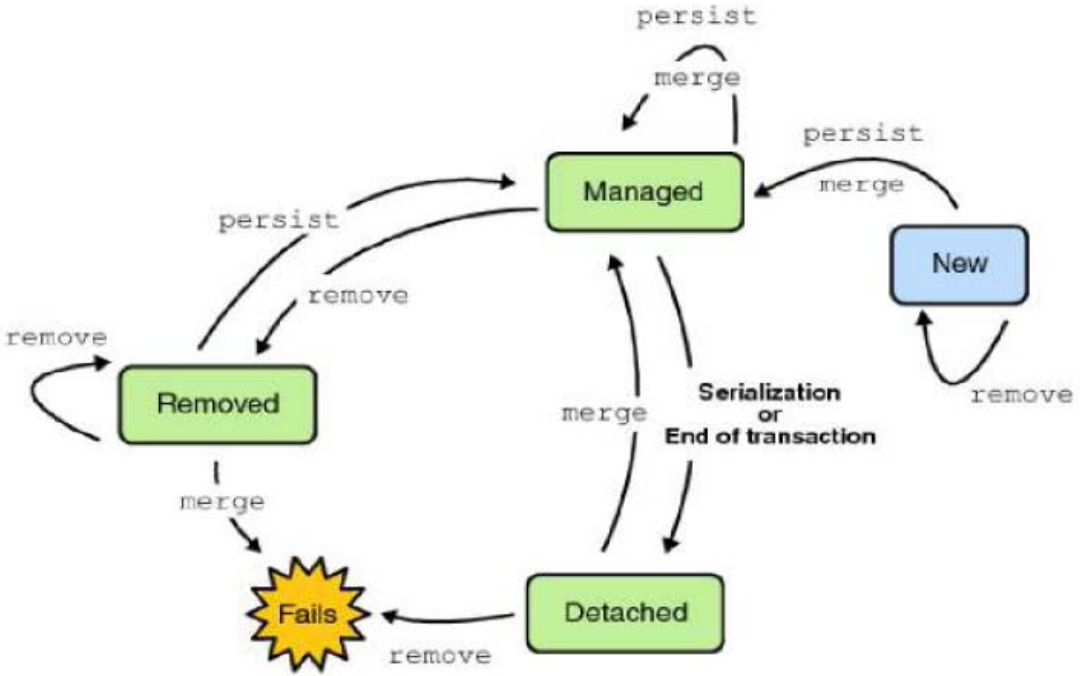
Ejemplo de Manejo de Clases Entidad.

```
import javax.persistence.*;
import javax.ejb.*;
@Remote @Stateless
public class BrokerModelImpl implements BrokerModel {
        @PersistenceContext private EntityManager em;
        public Stock getStock(String symbol) throws BrokerException {
                Stock stock = em.find(Stock.class, symbol);
                if(stock != null) {
                        return stock;
                } else {
                throw new BrokerException("Stock : " + symbol + " not found");
                }
        }
        public void addStock(Stock stock) throws BrokerException {
                try {
                        em.persist(stock);
                } catch(EntityExistsException exe) {
                        throw new BrokerException("Duplicate Stock : " +
        stock.getSymbol();
                }
        }
```

Ejemplo de Manejo de Clases Entidad (2).

```
public void updateStock(Stock stock) throws BrokerException {
        Stock s = em.find(Stock.class, stock.getSymbol());
        if(s == null) {
            throw new BrokerException("Stock : " + stock.getSymbol() + " not found");
        } else {
            em.merge(stock);
        }
        }
        public void deleteStock(Stock stock) throws BrokerException {
            String id = stock.getSymbol();
            stock = em.find(Stock.class, id);
            if(stock == null) {
                throw new BrokerException("Stock : " + stock.getSymbol() +" not found");
            } else {
                em.remove(stock);
        }
}
}
```

Ciclo de Vida de las Entidades.



CJAVA
siempre para apoyarte

Estados de las Entidades.

Estado	Significado
New	Una nueva instancia de la Entidad se creó usando new de Java. No hay un registro asociado a la llave primaria en la base de datos y la entidad no está conectada a un contexto de persistencia.
Managed	Hay un registro de la base de datos correspondiente y los datos se mantienen sincronizados. La entidad está conectada a un contexto de persistencia y tiene un identidad única. Solamente una instancia de una entidad "managed" puede existir en un contexto de persistencia.
Detached	Hay un registro de la base de datos correspondiente pero los datos no se sincronizan y la instancia de la entidad no está conectada a un contexto de persistencia.
Removed	Este estado representa que está pendiente el borrado del registro correspondiente en la base de datos.

CJAVA
siempre para apoyarte

Métodos del Entity Manager.

Método	Descripción
flush	Fuerza la sincronización de la base de datos con Entidades en el Contexto de Persistencia.
refresh	Refresca las Entidades del Contexto de la base de datos.
find	Encuentra una instancia ejecutando un query por llave primaria.
contains	Entrega verdadero si la Entidad está en el Contexto de Persistencia, es decir, está manejada.
merge	Actualiza el estado de la Entidad en el Contexto.
remove	Remueve la instancia de la Entidad.
persist	Graba la Entidad en la base de datos.

CJAVA
siempre para apoyarte

Anotaciones Especiales.

Una Entidad puede ser notificada antes o después de cambios en su ciclo de vida, utilizando métodos con anotaciones especiales:

- @PrePersist
- @PostPersist
- @PreRemove
- @PostRemove
- @PreUpdate
- @PostUpdate
- @PostLoad

CJAVA
siempre para apoyarte

Tópicos Avanzados de Persistencia.

- Relaciones – La API de Persistencia soporta el manejo de llaves foráneas y tablas relacionadas. En Java, estas relaciones se modelan mediante relaciones de Herencia de Entidades.
- Java Persistence Query Language – Es un dialecto de SQL portable utilizado para evitar dependencias de SQL en aplicaciones Java.
- Queries – de dos tipos:
  - NamedQueries – para Persistence Query Language.
  - NativeQueries. – para queries de SQL normal.

Ejemplo de un Query de Persitence Query Language.

```
@Entity
@NamedQuery(name="FindAllOpenAuctions",
query=" SELECT OBJECT(a) FROM Auction AS a WHERE a.status=
'OPEN'")
```

- Usan el Persistence Query Language en vez de SQL.
- Hacen la aplicación altamente portable.
- No pueden utilizar características especiales de SQL.

Ejemplo de un Query Nativo.

```
Query query = em.createNativeQuery("SELECT * FROM Customer",Customer.class);
List customers = query.getResultList();
Query query = em.createNativeQuery("SELECT * FROM SHARESWHERE
                    SSN = '" + customerId + "'", CustomerShare.class);
List shares = query.getResultList();
```

- Usan SQL standard.
- No se recomiendan porque pueden hacer la aplicación poco portable.

CJAVA

siempre para apoyarte

Gracias